

Chapter 4

Locks from the hardware up

A lock turns a critical section into something atomic. But how do you build a lock if every load and store can be interleaved with those of other threads? This chapter answers it from the bottom: the requirements, the one-CPU trick (disable interrupts), a classic software algorithm that fails on real hardware, the atomic instructions that make locks possible, and the choice between spinning and sleeping — which is exactly the difference between SWEB’s `SpinLock` and `Mutex`.

4.1 What a lock must guarantee

Mutual exclusion at most one thread in the critical section.

Progress if the lock is free and threads want it, one of them gets it (no deadlock among the contenders).

Bounded waiting a waiting thread gets its turn after a bounded number of others (no starvation) — “fairness”.

and in practice: cheap when uncontended, and not burning CPU time while waiting for long.

4.2 Disabling interrupts

On a single CPU, the only way another thread can run in the middle of your critical section is a timer interrupt that makes the scheduler switch. Turn interrupts off (`cli`), do the work, turn them back on (`sti`) — nothing can interrupt you. This is how kernels protected data before multi-core, and SWEB still uses it for the scheduler’s own data (`Scheduler::schedule` runs with interrupts off). But:

- it is privileged — user code cannot do it;
- it does not stop the *other* cores on a multi-core machine;
- with interrupts off, timer ticks and device interrupts are delayed; the section must be very short, and must not sleep.

4.3 Peterson’s algorithm — and why it breaks

With only loads and stores, two threads can still achieve mutual exclusion on paper:

```
int flag[2], turn;                /* shared */
void lock(int me) {
    int other = 1 - me;
    flag[me] = 1;                 /* I want in          */
    turn = other;                 /* but you go first   */
}
```

```

while (flag[other] && turn == other) {} /* wait while you want in and it's your turn */
}
void unlock(int me) { flag[me] = 0; }

```

It is correct under *sequential consistency* — the assumption that all memory operations happen in one global order that respects each thread’s program order. Real CPUs do not guarantee that. On x86, a store goes into the core’s *store buffer* and becomes visible to the other core later; a subsequent load from a *different* address can overtake it. Both threads store their `flag`, both load the other’s still-old `flag = 0` from memory, and both enter (module 04’s `peterson.c` shows the violation within seconds). A *memory fence* (`mfence`, `atomic_thread_fence(memory_order_seq_cst)`) between the stores and the loads repairs it. The lesson: correct synchronisation needs help from the hardware.

4.4 Atomic instructions

Test-and-set / exchange `old = xchg(&lock, 1)` writes 1 and returns the previous value, atomically. If `old` was 0, you have the lock. (`xchg` with memory is implicitly locked on x86.)

Compare-and-swap `CAS(&x, expected, new)`: if `x == expected`, write `new` — all at once — and report success. The universal building block: any read-modify-write can be written as a CAS loop (`lock cmpxchg`).

Fetch-and-add `old = fetch_add(&x, n)` (`lock xadd`): counters, and the ticket lock below.

C11 exposes them in `<stdatomic.h>`; GCC also has builtins such as `__sync_lock_test_and_set`, which SWEB uses.

4.5 Spinlocks

```

void spin_lock(atomic_int *l) {
    for (;;) {
        if (!atomic_exchange(l, 1)) return; /* test-and-set: got it */
        while (atomic_load(l)) {}           /* test: wait with plain reads */
    }
}
void spin_unlock(atomic_int *l) { atomic_store(l, 0); }

```

Waiting with plain reads (*test-and-test-and-set*) matters: every `xchg` needs exclusive ownership of the cache line, so a crowd of cores hammering `xchg` keeps moving the line between them and slows down even the lock holder. Reads can share the line until the holder’s unlock invalidates it.

A plain spinlock is unfair — whoever happens to win the race gets it. The **ticket lock** is FIFO: `my = fetch_add(&next, 1); while (serving != my) {}`; unlock increments `serving`. Like a deli-counter ticket.

4.6 Spin or sleep?

Spinning wastes the CPU for the whole wait. That is fine when the lock is held for a few instructions and the holder is running on another core. It is disastrous when the holder is *not running* — on a single CPU the spinner burns its entire time slice while the holder cannot make progress. Hence the three options:

1. spin (short critical sections, multi-core, holder never sleeps);
2. spin, but `yield` after a few tries (gives the holder a chance to run);

3. **sleep**: put yourself on a wait queue, block, and let the unlocker wake you. Costs two context switches, wastes no CPU. Linux's `futex` makes the uncontended case a single atomic instruction in user space and enters the kernel only to sleep and wake.

4.7 Memory ordering in one paragraph

Atomic operations also order the *other* memory accesses around them. Lock acquisition needs *acquire* semantics (nothing from the critical section may move before it), unlock needs *release* semantics (nothing may move after it). A release store paired with an acquire load that reads it creates a happens-before edge (chapter 3). C11's default, `memory_order_seq_cst`, is the strongest and simplest; use it unless you know exactly why a weaker order suffices.

In SWEB

`common/source/kernel/`: `SpinLock` tries `ArchThreads::testSetLock` (an `xchg`); if the lock is taken it records itself as waiting and loops *test-and-set + yield* — spinning without yielding would be pointless on SWEB's single CPU. `Mutex` sleeps instead: on contention it enqueues the thread in its waiters list and calls `Scheduler::sleep()`; `release` pops a waiter and wakes it. Both inherit from `Lock`, which keeps the per-thread lists used for deadlock checks (chapter 6). User-space locks in SWEB (`pthread_spin_*`, and mutexes that sleep only through a small system call) are part of P1's elective.

Key points

- Requirements: mutual exclusion, progress, bounded waiting.
- Disabling interrupts works only on one CPU and only in the kernel.
- Peterson needs sequential consistency; store buffers break it without fences.
- Locks are built from atomic read-modify-write instructions (`xchg`, `CAS`, `fetch-and-add`).
- Spin briefly, otherwise sleep; on one CPU never spin without yielding.
- Lock = acquire, unlock = release: that is what makes the data written inside visible to the next holder.